

FORMATION EMBARQUÉE

Projets d'évaluation

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Projets d'évaluation finale (avec barème)

Trois sujets « examen », un par grande voie. Choisis-en **un** en fin de parcours (ou fais les trois à quelques mois d'écart). Conditions : partir de zéro, documenter comme si un collègue devait reprendre le projet, et tenir un journal de bord (une entrée par session de travail). Durée indicative : 30-40 h chacun.

Sujet A — Firmware : enregistreur de données autonome (STM32)

Cahier des charges

Un enregistreur de mesures environnementales sur batterie :

1. Mesure T°/humidité/pression (BME280, driver **maison**) toutes les période configurables (10 s à 1 h).
2. Horodatage par **RTC interne** du STM32 (réglable par la console).
3. Stockage en **flash SPI externe ou EEPROM I2C** (driver maison aussi), format binaire compact documenté, à l'épreuve des coupures (un enregistrement à moitié écrit ne corrompt pas les autres).
4. **Console série** : `status` , `dump [n]` (relit les n derniers enregistrements en CSV), `set period` , `set time` , `erase` (avec confirmation).
5. **Basse consommation** : mode Stop entre deux mesures, réveil par RTC ; mesurer et documenter la consommation estimée (fiche de calcul d'autonomie sur pile CR2032 ou batterie LiPo).
6. Robustesse : capteur débranché, flash pleine (politique circulaire documentée), montre jamais réglée — tout est géré et signalé.

Barème (/100)

Critère	Points
Drivers BME280 + mémoire : propres, sans dépendance, erreurs gérées	20
Format de stockage documenté + intégrité sur coupure (test à l'appui)	15
Console complète et robuste (entrées hostiles testées)	10
Basse consommation mise en œuvre + calcul d'autonomie argumenté	15
Architecture (modules, zéro accès matériel hors drivers, FSM claires)	15
Qualité C : -Wall -Wextra propre, types stdint, pas de malloc, asserts	10
Git (historique lisible) + README (schéma, photos, mode d'emploi)	10
Journal de bord honnête (impasses comprises)	5

Seuil de réussite : 70. Excellence (90+) : ajout d'un watchdog documenté, tests unitaires des drivers compilés sur PC (mock du bus I2C).

Sujet B — FPGA : chronomètre d'affichage multiplexé (VHDL)

Cahier des charges

Sur simulateur (GHDL) puis carte si disponible :

1. Chronomètre MM:SS.CC (centièmes), boutons start/stop/reset (anti-rebond **matériel** du TD 04).
2. Affichage sur **4 afficheurs 7 segments multiplexés** (rafraîchis à ~1 kHz par balayage — le module de multiplexage est un composant réutilisable avec generics).
3. **Mémoire de 4 temps intermédiaires** (bouton « lap ») relus par un 5^e bouton, avec indication du numéro affiché.
4. Sortie **UART TX** (ton module du TP 2) : chaque « lap » émet le temps en ASCII (`01:23.45\r\n`).
5. Toute la conception **synchrone à une seule horloge**, entrées synchronisées, zéro latch (le rapport de synthèse en atteste si carte).

Barème (/100)

Critère	Points
Découpage hiérarchique (diviseurs, compteurs BCD en cascade, mux 7 seg, FSM boutons, UART)	20
Testbenchs : un par module + un testbench top avec scénario complet	25
Zéro latch, zéro signal multi-piloté, synchroniseurs présents	15
Chronomètre juste (vérifié au chronogramme : 1 centième = N cycles exact)	10
Laps : mémorisation/relecture correctes (cas « 5 ^e lap » défini et testé)	10
UART conforme (trame vérifiée au chronogramme)	10
Compte rendu : schéma bloc, chronogrammes annotés, journal	10

Seuil : 70. Excellence : contraintes + passage sur carte réelle avec photo, ou ajout d'un réglage de l'heure via UART RX.

Sujet C — Automatisme : machine de conditionnement (TIA Portal ou EcoStruxure)

Cahier des charges

Une machine emballe des produits par lots :

1. Convoyeur d'amenée, détection produit, **poussoir** à vérin (capteurs sorti/rentre) qui transfère le produit vers la zone de cartonnage.
2. Un carton = N produits (N réglable 4-24). Carton plein → tapis d'évacuation 5 s → nouveau carton (signal « carton présent » requis).
3. Modes AUTO / MANU (commandes unitaires sécurisées) / **REGLAGE** (vérin pas à pas, réservé au niveau « régleur » de l'HMI).

4. Défauts gérés : AU (NF), vérin qui n'atteint pas son capteur en 3 s (**surveillance de mouvement**), absence carton, bourrage cellule. Chaque défaut : arrêt sûr adapté (le vérin **ne revient pas tout seul** si un produit est engagé — à justifier), voyant, alarme HMI, acquit.
5. HMI : conduite (synoptique animé, compteurs lot/total), réglages (bornés, par niveau), page alarmes avec historique.
6. **Table Modbus/OPC documentée** (10 mots min. avec mot de vie) + un client PC (Python/Java) qui journalise la production en CSV.

Barème (/100)

Critère	Points
Dossier d'analyse AVANT code : E/S, GRAFCET (conduite + surveillance), analyse de défauts	25
Structure blocs : séquence / défauts / modes / sorties séparés, sorties centralisées	15
Surveillances de mouvement et positions de repli justifiées	15
Recette de test écrite et déroulée (≥ 10 scénarios dont 4 de défaut)	15
HMI complète (impulsions, bornes, niveaux, alarmes)	10
Table d'échange + client PC avec mot de vie	10
Journal + captures du déroulé de recette	10

Seuil : 70. Excellence : la même application portée sur le **deuxième** environnement (TIA ↔ EcoStruxure) avec une note comparant les deux — c'est un excellent sujet d'entretien.

Comment t'auto-évaluer honnêtement

1. Note chaque ligne du barème **avec une preuve** (capture, extrait de code, ligne du journal). Pas de preuve = pas les points.
2. Laisse reposer une semaine, puis rejoue TA recette de test depuis zéro (environnement fraîchement ouvert). Ce qui casse ta note mieux que toi.
3. Publie le projet sur GitHub avec le barème rempli dans le README : l'exercice de transparence est en soi une compétence professionnelle — et un excellent signal en entretien.